

RAPPORT D'INGÉNIERIE TECHNIQUE

Mission SENTINEL-X

Nœud IoT ESP8266, stack Docker sécurisée, IA prédictive et vision embarquée

Cadre	EPSI — Workshop Bac+4 (M1), octobre 2026
Option technique	Option B — topologie « Edge-to-Server » : le PC d'un apprenant joue le rôle de serveur local
Groupe	M1-G__
Version du document	6 octobre 2026 — dépôt jeudi 8 octobre 2026 (rapport d'audit complété après le pentest)
Fichier	Workshop2026-M1-G__-Dossier.pdf
Code source	https://github.com/Guiggzz/SENTINEL-X

ÉQUIPE (CONSORTIUM)

Membre	Filière (DEV / IA / INFRA / CYBER)	Rôle principal
Nathanaël Lejuste	DEV	API
Rémy Eroes	DEV	Dashboard & système embarqué
Amaury Malisova	DEV	IA
Guillaume Breon	DEV	Dashboard & système embarqué

00 Sommaire et synthèse

01	Architecture et schéma réseau	4
02	Schéma de câblage électronique ESP8266	7
03	Matrice de sécurité — TLS, hardening, preuves	8
04	Documentation de l'IA — maintenance prédictive, vision, reconnaissance faciale	11
05	Rapport d'audit post-pentest (jeudi 08/10)	15
A3	Annexe — poster A3 paysage	17

Synthèse

SENTINEL-X est un module de surveillance de table pour les centrales AetherCorp. Un nœud ESP8266 mesure la température, l'humidité, le gaz/fumée et la présence, affiche son état sur un écran OLED et pilote un buzzer et deux LED. Il publie ses mesures en MQTT chiffré (MQTTS) vers une stack Docker hébergée sur le PC portable d'un membre de l'équipe (Option B). Cette stack stocke les données dans PostgreSQL, les expose sur un tableau de bord HTTPS temps réel, et alimente un service d'IA qui détecte les dérives anormales des capteurs sans seuil statique. Une webcam USB reliée au PC alimente le service de vision : YOLOv8n détecte les personnes, puis un portillon d'identification demande à la personne de s'identifier (bips et écran OLED) et la reconnaît par son visage (YuNet + SFace, contrôle de vivacité MiniFASNet). Une personne non identifiée en 5 s est journalisée et signalée sur Discord avec sa photo ; la sirène ne sonne que si l'« Alarme reconnaissance » est armée.

Conteneurs Docker	Flux chiffrés	Inférence YOLO / trame	Rappel spray butane	Fenêtre d'identification
5	TLS 1.3	≈ 42 ms	20/20	5 s

Brique	Technologie	Emplacement
Nœud capteurs	ESP8266 NodeMCU v3, Arduino C++, PubSubClient + BearSSL	firmware/sentinel_node/
Broker	Eclipse Mosquitto 2.0.20, listener unique 8883 (TLS, ACL, anonyme refusé)	infra/mosquitto/
API + dashboard	FastAPI, WebSocket /ws, session opérateur, Bearer pour les clients machine	backend/
Base de données	PostgreSQL 16 (volume nommé, aucun port publié)	infra/docker-compose.yml
IA prédictive	Isolation Forest (scikit-learn), 8 features cinétiques	ml/
Vision	Ultralytics YOLOv8n sur CPU, flux MJPEG, service systemd utilisateur	vision/
Reconnaissance faciale	OpenCV YuNet (détection) + SFace (empreinte), anti-spoofing MiniFASNet, portillon d'identification	vision/face_*.py, identity.py
Alerte externe	Webhook Discord (HTTPS sortant), photo de la personne non identifiée	vision/discord_alert.py
Reverse proxy	Caddy 2.8, HTTPS 443, redirection 80 → 443, en-têtes de sécurité (HSTS, CSP stricte, X-Frame-Options)	infra/caddy/Caddyfile

Livrables

Livrable	Contenu
Workshop2026-M1-G__-Dossier.pdf	le présent rapport : schéma réseau, câblage, matrice de sécurité, IA, rapport d'audit post-pentest, poster A3 en annexe

Workshop2026-M1-G__-Pres.pptx	support de la soutenance du vendredi 9 octobre
Workshop2026-M1-G__-VidDrop.mp4	vidéo verticale « Sentinel Drop » (60 s)
Code source	dépôt GitHub https://github.com/Guiggzz/SENTINEL-X (README détaillé)

Aucun secret n'est versionné : `firmware/sentinel_node/secrets.h`, `infra/.env`, `infra/OPERATOR_PASSWORD.txt`, les fichiers de mots de passe Mosquitto, les clés privées TLS et le mot de passe OTA sont exclus par `.gitignore` (aucun de ces fichiers dans l'historique git, contrôle du 6 octobre 2026). L'URL du webhook Discord est stockée hors du dépôt.

Toutes les valeurs chiffrées de ce rapport proviennent du dépôt (README, `ml/evaluation.md`, fichiers d'infrastructure, code du firmware et de la vision) ou de mesures relevées sur la machine les 5 et 6 octobre 2026.

01 Architecture et schéma réseau

Option B : le PC apprenant (Ubuntu, interface Wi-Fi `wlp0s20f3`) héberge toute la stack. Le boîtier ne contient que l'ESP8266 et ses capteurs. La webcam est branchée en USB sur le PC.

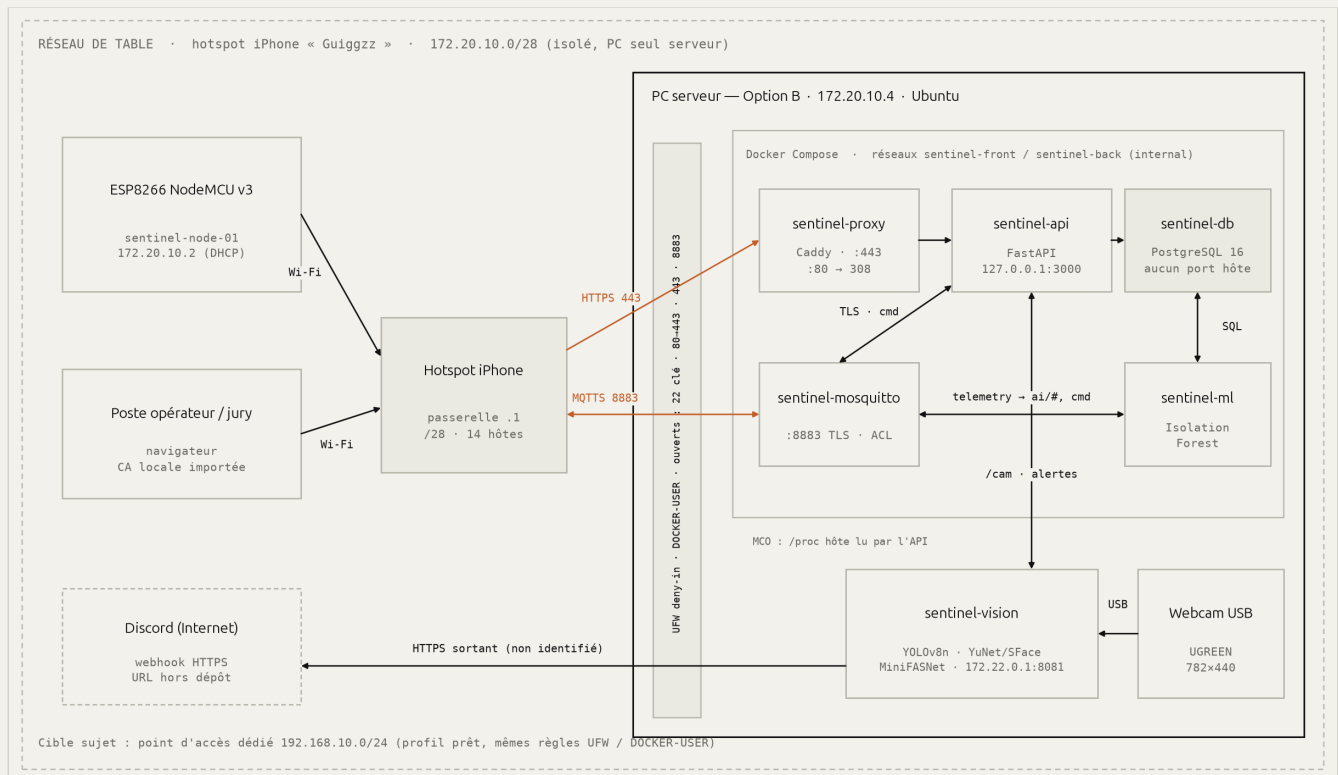


Fig. 1 – Topologie de table et flux. Accent : flux chiffrés entrants autorisés par le pare-feu ; pointillés : service externe.

Plan d'adressage

Élément	Actuel (hotspot iPhone « Guiggzz »)	Cible sujet (point d'accès dédié) — profil prêt
Sous-réseau	172.20.10.0/28 (14 hôtes utiles)	192.168.10.0/24 (SSID SENTINEL-X)
Passerelle / AP	172.20.10.1 (iPhone)	192.168.10.1 (PC en mode AP)
PC serveur	172.20.10.4 (DHCP)	192.168.10.1
ESP8266 sentinel-node-01	172.20.10.2 (observé dans les journaux Mosquitto)	pool DHCP .10 – .50
Règle MQTTS	8883 autorisé depuis 172.20.10.0/28 uniquement	même règle, déjà ouverte pour 192.168.10.0/24

POURQUOI UN HOTSPOT PLUTÔT QUE LE POINT D'ACCÈS DÉDIÉ

Le sujet demande un point d'accès dédié et un sous-réseau de table étanche. Le profil NetworkManager **SENTINEL-X-AP** (192.168.10.0/24, DHCP .10 – .50) est prêt et les règles UFW / DOCKER-USER l'autorisent déjà, mais il n'est pas activé : le portable n'a qu'une interface Wi-Fi, et la passer en mode AP coupe le hotspot, seul accès Internet de l'équipe (docs/reseau-table.md). Le hotspot iPhone remplit le même rôle d'isolation : il n'héberge que les équipements de la table, son /28 limite le réseau à 14 hôtes, et le PC y est le seul serveur. Les ports exposés sont filtrés par UFW et, pour les ports publiés par Docker, par la chaîne DOCKER-USER (8883 n'est accepté que depuis les sous-réseaux de table). La bascule vers l'AP ne demande que l'activation du profil et le changement de MQTT_HOST dans le firmware.

Matrice des flux

Source → destination	Port / protocole	Exposition	Protection
ESP8266 → sentinel-mosquitto	8883 MQTTS	LAN, sous-réseau de table	TLS, pin SHA1 du certificat serveur, utilisateur dédié + ACL
Navigateur → sentinel-proxy	443 HTTPS	LAN	TLS 1.2/1.3 (CA locale), session HttpOnly/Secure/SameSite=Strict, contrôle d'Origin
Navigateur → sentinel-proxy	80 HTTP	LAN	aucun contenu : redirection 308 vers HTTPS
Administration → PC	22 SSH	LAN	clé uniquement (mot de passe et root refusés), UFW limit
sentinel-proxy → sentinel-api	3000 HTTP	réseau Docker	publié seulement sur 127.0.0.1, UFW deny 3000
sentinel-api → sentinel-vision	8081 HTTP	172.22.0.1 (passerelle Docker)	pas joignable depuis le Wi-Fi ; Bearer pour /faces ; accès client via /cam/* authentifié
sentinel-vision → sentinel-api	3000 HTTP	hôte local	POST /api/v1/alerts avec Bearer API_TOKEN
sentinel-vision → Discord	443 HTTPS sortant	Internet (hotspot)	URL du webhook hors dépôt ; photo envoyée seulement si non identifié ; 1 envoi / 30 s au plus
api / ml → sentinel-db	5432	réseau sentinel-back (internal)	aucun port hôte
api / ml → sentinel-mosquitto	8883 MQTTS	réseau Docker	TLS + comptes séparés (sentinel-ml écrit sentinel/ai/# et sentinel/+/cmd)

Flux d'identification. Service vision → API (POST /api/v1/alerts, états identify_start, identified, intrusion...) → MQTT sentinel/sentinel-node-01/cmd → ESP (bips, écran OLED, sirène). En parallèle, la vision envoie directement la photo à Discord quand la personne n'est pas identifiée (détail en §4.4).

Topics MQTT

Topic	Sens	Contenu
sentinel/{device_id}/telemetry	ESP → broker	JSON capteurs (température, humidité, gaz, présence, actionneurs, RSSI)
sentinel/{device_id}/alerts	ESP → broker	événements présence / actionneurs
sentinel/{device_id}/status	ESP → broker (retained)	online / offline
sentinel/{device_id}/cmd	API, ML → ESP	buzzer (songs gaz, intrus...), bips, LED, mode auto, écran OLED (display : identify / authorized / intrusion / normal)
sentinel/ai/<device>/risk · ai/alerts · ai/status	ML → broker	risque 0-100, incidents IA, LWT du service

Services Docker

Conteneur	Image	Ports hôte	Réseaux	Limites
sentinel-mosquitto	eclipse-mosquitto:2.0.20	8883	front, back	128 Mo · 0,5 CPU · 100 PID
sentinel-db	postgres:16-alpine	aucun	back	512 Mo · 1 CPU · 200 PID
sentinel-api	build backend/	127.0.0.1:3000	front, back	512 Mo · 1 CPU · 200 PID

sentinel-ml	build ml/	aucun	back	768 Mo · 1 CPU · 200 PID
sentinel-proxy	caddy:2.8-alpine	80 (→ 308) · 443	front	128 Mo · 0,5 CPU · 100 PID

Journaux Docker en rotation (`json-file`, 10 Mo × 3 fichiers par conteneur) pour éviter la saturation du disque sous l'afflux de messages MQTT. Le bloc MCO du dashboard (`/api/v1/system`) suit CPU, RAM et disque de l'hôte, le débit MQTT et l'état des sondes mosquitto / db / ml / nœud.

02 Schéma de câblage électronique ESP8266

Carte NodeMCU v3 alimentée en USB. Affectation des broches relevée dans `firmware/sentinel_node/sentinel_node.ino` : « DHT22 D5 | PIR D6 | MQ-2 A0 | OLED SDA D2 / SCL D1 | Buzzer D7 | LED rouge D0 | LED verte D8 ».

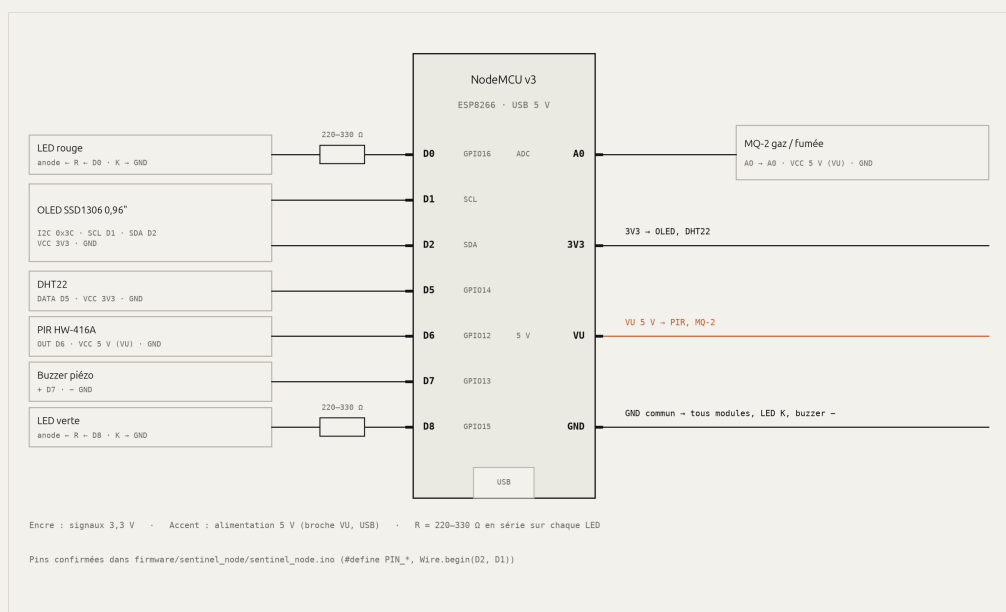


Fig. 2 – Câblage du nœud sentinel-node-01.

Composant	Broche ESP	Alimentation	Remarque
DHT22 (température / humidité)	D5 (GPIO14)	3V3 · GND	lecture toutes les 2 s (limite du capteur)
OLED SSD1306 0,96" I2C	D1 SCL · D2 SDA	3V3 · GND	adresse 0x3C, <code>Wire.begin(D2, D1)</code> ; affiche IP, état MQTTS, alertes ; écrans du portillon (commande <code>display</code>)
PIR HW-416A	D6 (GPIO12)	5 V (VU) · GND	sortie logique 3,3 V ; anti-rebond logiciel
MQ-2 gaz / fumée	A0 (sortie A0)	5 V (VU) · GND	chauffe nécessaire après mise sous tension
Buzzer piézo	D7 (GPIO13)	– · GND	sirènes gaz et intrus (<code>alarms.h</code>), bips « identifiez-vous »
LED rouge	D0 (GPIO16)	via R 220–330 Ω → GND	présence, alarme ; clignote si déconnecté
LED verte	D8 (GPIO15)	via R 220–330 Ω → GND	connecté, RAS ; « accès autorisé »

Points d'attention

- **D8 / GPIO15** doit être au niveau bas au démarrage : la LED et sa résistance vers GND jouent le rôle de pull-down, ne pas y brancher de pull-up.
- **A0** : l'entrée analogique de la NodeMCU accepte 0–3,3 V grâce à son pont diviseur. Le MQ-2 étant alimenté en 5 V, vérifier que la tension AO reste dans cette plage (un pont diviseur externe sinon).
- **Masse commune** à tous les modules ; le 5 V est pris sur VU (USB). **Cadence** : le firmware actuel publie la télémétrie toutes les 0,5 s (`TELEMETRY_MS`), le DHT22 n'est relu que toutes les 2 s ; le modèle IA raisonne sur une période nominale de 2 s.
- **PIR** : jugé instable sur table, il reste publié dans la télémétrie ; l'alarme intrusion s'appuie sur la vision (YOLO et portillon).

03 Matrice de sécurité

Sources : docs/matrice-securite.md, docs/preuves-securite.txt (durcissement avant pentest du 6 octobre 2026, 09:52), journal de infra/harden-root.sh (6 octobre 2026, 10:24) et contrôles en lecture seule du 6 octobre 2026 vers 11:40 (infra/hardening/check-demo.sh, ss, curl, docker inspect).

Menace	Vecteur (jury / pentest)	Impact	Mitigation implémentée	Risque résiduel
MitM / sniffing MQTT	Wireshark sur le hotspot	Lecture télémetrie, injection de commandes	MQTTs seul (8883, TLS 1.2 minimum, négocié en TLS 1.3), 1883 fermé ; ESP : pin SHA1 du certificat (MQTT_CERT_FINGERPRINT)	Certificat auto-signé (CA locale à importer)
Injection de payload MQTT	Client rogue / Metasploit	Commandes ESP, fausses alertes	allow_anonymous false, client-id vide refusé, ACL par utilisateur (le nœud n'écrit que ses 3 topics et ne lit que son cmd) ; /commands en liste blanche (actions, mélodies, durées, device_id filtré)	Compromission d'un secret MQTT
DoS broker / proxy	Flood connexions / messages	Perte de la supervision	max_connections 50, max_inflight_messages 20, max_queued_messages 100, max_packet_size 8192, max_queued_bytes 1 Mo ; Caddy : timeouts, corps limité ; limites CPU/RAM/PID	DoS Wi-Fi (L2/L3) hors MQTT
Brute force login	POST /auth/login	Prise de contrôle du dashboard	5 échecs / 15 min par IP → verrouillage 15 min, 10 requêtes/min ; journal LOGIN_FAIL / LOGIN_LOCKOUT	Verrouillage en mémoire (remis à zéro au redémarrage de l'API)
Vol de session, CSRF, XSS	Page tierce, script injecté	Commandes au nom de l'opérateur	Cookie HttpOnly + Secure + SameSite=Strict, 4 h, révoqué au logout ; Origin/Referer obligatoire sur les requêtes par cookie (403 sinon) ; anti-CSWSH sur /ws ; CSP stricte sans unsafe-inline, X-Frame-Options DENY	Sessions en mémoire
Exposition de services	Nmap	Surface d'attaque	UFW verrouillé (deny par défaut) ; ouverts : 22, 80, 443, 8883 ; Samba et Apache arrêtés ; conteneurs healthai-* arrêtés ; DOCKER-USER pour 8883	Retour d'un service ou d'un bind 0.0.0.0 (recontrôle avant pentest)
Accès caméra	URL directe du flux	Fuite d'images	Service vision lié à 172.22.0.1:8081 (passerelle Docker) ; flux servi via /cam/* derrière le cookie de session (401 sans session)	—
Usurpation de visage	Photo ou écran devant la webcam	Accès « autorisé » indu	Vivacité MiniFASNet : un visage connu n'est accepté que s'il est vivant ; leurre = intrusion	Attaques de haute qualité (masque, vidéo plein écran)
Accès physique ESP	Vol, dump flash	Secrets Wi-Fi / MQTT	secrets.h hors git, utilisateur MQTT dédié restreint par ACL	Flash lisible avec accès physique
Évasion de conteneur	CVE / mauvaise config Docker	Accès hôte	cap_drop ALL partout, no-new-privileges, read_only (api, ml, proxy), api/ml non-root, limites	Docker rootful

État du durcissement

Mesure	État	Preuve / commentaire
MQTTS 8883, port 1883 fermé	● FAIT	TLS 1.3 négocié, TLS 1.2 accepté (<code>check-demo.sh</code>) ; connexion 1883 refusée ; seul listener 8883 dans <code>mosquitto.conf</code> , anonyme refusé
ESP en MQTTS avec pin du certificat	● FAIT	<code>setFingerprint(MQTT_CERT_FINGERPRINT)</code> ; connexions <code>sentinel-node-01</code> depuis 172.20.10.2 côté broker. Pin SHA1 : fonctionne hors ligne sans NTP.
HTTPS 443 et redirection 80	● FAIT	<code>http://127.0.0.1/</code> → 308 vers HTTPS (Caddy) ; TLS 1.2/1.3 seulement ; HSTS 1 an, CSP stricte, X-Frame-Options DENY, nosniff, Referrer-Policy no-referrer
Authentification, anti brute-force, CSRF	● FAIT	8 mots de passe faux : 5 × 401 puis 429, même le bon mot de passe reste bloqué depuis l'IP verrouillée, une autre IP passe ; POST avec Origin étranger → 403 (preuves du 6 octobre 2026)
Caméra derrière authentification	● FAIT	<code>/cam/snapshot.jpg</code> sans cookie → 401, avec cookie → 200 (<code>check-demo.sh</code> , 6 octobre 2026, 11:40)
UFW	● VERROUILLÉ	deny en entrée et en routage ; 22 en <code>limit</code> , 80, 443, 8883 depuis 172.20.10.0/28 et 192.168.10.0/24 ; refus explicites 3000, 5432, 8081 (hors réseau Docker), 139/445, 137/138
DOCKER-USER	● FAIT	8883 accepté seulement depuis les sous-réseaux de table (IPv4), bloqué en IPv6 ; bloc persistant dans <code>/etc/uw/after.rules</code>
Samba, Apache	● ARRÊTÉS	<code>smbd</code> , <code>nmbd</code> , <code>apache2</code> : inactive et disabled ; le port 80 est repris par Caddy pour la seule redirection
Conteneurs tiers healthai-*	● ARRÊTÉS	4 conteneurs arrêtés, politique <code>restart=no</code> ; 3002, 3100 et 9090 ne sont plus en écoute
Conteneurs durcis	● FAIT	<code>cap_drop ALL</code> sur les 5 conteneurs (ajouts minimaux : <code>mosquitto</code> CHOWN/SETUID/SETGID, <code>db</code> CHOWN/DAC_OVERRIDE/FOWNER/SETUID/SETGID, <code>proxy</code> NET_BIND_SERVICE), <code>no-new-privileges</code> ; <code>api</code> , <code>ml</code> , <code>proxy</code> en <code>read_only</code> ; <code>api/ml</code> non-root. <code>mosquitto</code> et <code>db</code> restent en écriture (entrypoints officiels).
SSH par clé uniquement	● FAIT	<code>AuthenticationMethods</code> <code>publickey</code> , <code>PasswordAuthentication</code> <code>no</code> , <code>PermitRootLogin</code> <code>no</code> , <code>MaxAuthTries</code> 3 (<code>00-sentinel-hardening.conf</code>)
sysctl réseau	● FAIT	<code>rp_filter</code> , source routing et redirections refusés, <code>log_martians</code> , <code>kptr_restrict</code> , <code>dmesg_restrict</code>
Contrôle des ports	● FAIT	<code>ss -tln</code> (6 octobre 2026, 11:40) : hors loopback, seuls 22, 80, 443 et 8883 (8081 uniquement sur 172.22.0.1). Scan Nmap depuis une autre machine de la table à rejouer jeudi matin.
Docker rootless / usersns-remap	● NON FAIT	recommandation, nécessite de reconfigurer dockerd hors démo
fail2ban	● NON FAIT	non installé ; compensé par UFW <code>limit</code> sur 22 et le verrouillage applicatif du login

Surface exposée et risques résiduels

Port	Service	Protection
22/tcp	OpenSSH	clé uniquement, root refusé, UFW <code>limit</code>
80/tcp	Caddy (sentinel-proxy)	redirection 308 vers HTTPS, aucun contenu
443/tcp	Caddy → API	TLS 1.2/1.3, session + contrôle d'Origin, CSP stricte

8883/tcp

Mosquitto

TLS, comptes + ACL, accepté seulement depuis le sous-réseau de table ; ESP avec pin du certificat

- **Certificat auto-signé** : les navigateurs ignorent HSTS tant que la CA locale n'est pas importée.
- **Accès physique à l'ESP** : la flash est lisible ; elle contient les secrets Wi-Fi et MQTT du nœud (compte limité par ACL).
- **Verrouillage du login et sessions en mémoire** : remis à zéro au redémarrage de l'API.
- **Anti-spoofing** : modèle 2D passif, il peut être trompé par une attaque de haute qualité (masque 3D, vidéo plein écran sans bord visible, grande impression bien éclairée).
- **Docker rootful** et **DoS radio** sur le Wi-Fi : hors de portée des mesures logicielles.

Extraits de preuves

```
### infra/hardening/check-demo.sh — 6 octobre 2026, 11:40
[OK] API /health : {"status":"ok","mqtt_connected":true,"database":"ok"}
[OK] login via https://localhost/ -> 200
[OK] dashboard https://localhost/ (cookie) -> 200
[OK] API /telemetry/latest (cookie) -> 200
[OK] /cam/snapshot.jpg (cookie) -> 200 image/jpeg 53237
[OK] /cam/snapshot.jpg sans cookie -> 401
[OK] MQTT 8883 accepte TLS 1.2

### Ports en écoute hors loopback (ss -tln)
0.0.0.0:22  0.0.0.0:80  0.0.0.0:443  0.0.0.0:8883  172.22.0.1:8081
[::]:22    [::]:80    [::]:443    [::]:8883

### curl http://127.0.0.1/          http80=308 -> https://127.0.0.1/
### Port 1883                      connexion refusée
### healthai-*                     restart=no running=false (x4)

### harden-root.sh — UFW status (6 octobre 2026, 10:24)
Default: deny (incoming), allow (outgoing), deny (routed)
22/tcp      LIMIT IN  Anywhere      # SSH cle uniquement (rate-limit)
80/tcp      ALLOW IN  Anywhere      # SENTINEL redirection HTTP->HTTPS (Caddy)
443/tcp     ALLOW IN  Anywhere      # SENTINEL HTTPS
8883/tcp    ALLOW IN  172.20.10.0/28 # SENTINEL MQTTS ESP
8883/tcp    ALLOW IN  192.168.10.0/24 # SENTINEL MQTTS ESP
3000/tcp    DENY IN  Anywhere      # SENTINEL API interne
5432/tcp    DENY IN  Anywhere      # PostgreSQL interne
139,445/tcp DENY IN  Anywhere      # Samba interdit
```

Preuve Wireshark à réaliser sur le hotspot pendant la démo : filtre `tcp.port==8883`, charge utile illisible (TLS).

04 Documentation de l'IA

4.1 Maintenance prédictive — Isolation Forest

Le service `sentinel-ml` ne contient aucune règle du type `if temp > 40`. Un modèle **Isolation Forest** (scikit-learn, 300 arbres, contamination 0,005) apprend la dynamique normale des capteurs et signale les anomalies cinétiques : dérives, montées rapides, corrélations suspectes. Les seuils de décision sont **appris** à partir des quantiles du score sur l'historique normal, et non fixés sur une valeur capteur.

FEATURES (ML/FEATURES.PY)

Feature	Fenêtre	Ce qu'elle capte
gas_dev_rel	ligne de base EWMA $\approx$ 5 min	écart relatif du gaz à sa ligne de base (micro-déviations, fuite)
gas_slope_30s	30 s	pente du gaz (spray butane : montée brutale)
gas_std_30s	30 s	turbulence du panache
gas_slope_2m	2 min	dérive lente du gaz
temp_slope_2m	2 min	échauffement lent ($^{\circ}\text{C}/\text{min}$)
temp_dev	ligne de base EWMA $\approx$ 10 min	écart de température
hum_slope_2m	2 min	variation d'humidité (assèchement lors d'un échauffement)
gas_temp_corr	2 min	corrélations gaz $\times$ température : hausse lente de T° et micro-déviations de gaz

Features directionnelles. L'Isolation Forest est symétrique par nature. Avant le modèle, `vectorize()` ne conserve que le sens du danger (hausse de gaz, échauffement, chute d'humidité, corrélation gaz-température positive) et met à zéro les baisses et les retours à la normale. Une décroissance bénigne, comme le gaz qui redescend après un essai, n'est plus prise pour une anomalie. Autres garde-fous : filtre médian sur 3 échantillons contre les glitches ADC ; ligne de base presque gelée ($\times 0,05$) pendant une anomalie pour ne pas « absorber » une fuite ; réinitialisation de l'état au redémarrage de l'ESP (chauffe du MQ-2).

DÉCISION ET ACTIONS

- Score d'anomalie = `-score_samples()` sur features standardisées.
- Risque 0-100 interpolé entre la médiane normale (0), la vigilance q98 (30), le seuil d'anomalie appris (50) et un incident extrême de référence (100).
- Type d'incident selon la contribution des features (z-scores robustes médiane/MAD) : groupe gaz $\rightarrow$ `gaz_fumee`, groupe thermique $\rightarrow$ `thermique`, sinon `anomalie`.
- Persistance : 4 fenêtres anormales consécutives ($\approx$ 8 s) pour lever l'incident, 5 normales pour le clore.
- Prédiction (pré-alerte sans son) si le risque lissé monte en zone de vigilance dans le sens d'un danger.

État	Action
prediction	alerte <code>ia_prediction/risque_croissant</code> sur le journal et le dashboard, sans son
gaz_fumee	alerte <code>ia_gaz</code> , sirène <code>gaz</code> en boucle, LED rouge fixe ; réarmement 60 s après « Couper alarme » si le gaz persiste
thermique	alerte <code>ia_thermique</code> , LED rouge, bandeau dashboard
anomalie	alerte <code>ia_anomalie/anomalie_cinetique</code> (journal et dashboard)

ÉVALUATION (ML/EVALUATION.MD, GÉNÉRÉ LE 5 OCTOBRE 2026 À 14:56, GRAINE 42)

Entraînement sur 1 004 fenêtres réelles de `sentinel-node-01` (5 octobre 2026, 11:40 $\rightarrow$ 14:14, heure de Paris) et 4 016 fenêtres augmentées (bruit MQ-2 ± 10 ADC, DHT22 $\pm 0,5$ $^{\circ}\text{C}$ / ± 2 %RH, scénarios « personne proche » traités

comme normaux). Les essais réels (briquet, chauffage du DHT22, capteur débranché, chauffes après reboot) sont exclus via `ml/training_exclusions.json`. Split temporel 80 / 20.

Rappel spray butane (≤ 12 s) 20/20	Latence alarme 8 s médiane · max 10 s	Faux incidents (holdout) 0 192 fenêtres · 0,11 h	Glitches isolés ignorés 20/20	Dérives lentes avant règle 4/4
--	--	---	---	--

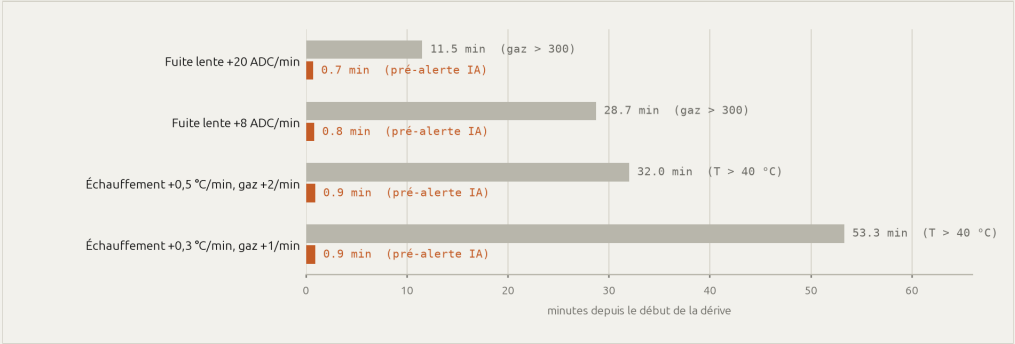


Fig. 3 — Dérives lentes simulées : première pré-alerte IA comparée au franchissement d'une règle statique (`ml/evaluation.md §3`).

Scénario	Pré-alerte IA	Incident confirmé	Règle statique	Avance IA
Fuite lente +20 ADC/min	0,7 min	0,8 min (gaz_fumee)	11,5 min (gaz > 300)	10,8 min
Fuite lente +8 ADC/min	0,8 min	non	28,7 min (gaz > 300)	27,9 min
Échauffement +0,5 °C/min, gaz +2 ADC/min	0,9 min	6,2 min (thermique)	32,0 min (T > 40 °C)	31,1 min
Échauffement +0,3 °C/min, gaz +1 ADC/min	0,9 min	6,7 min (anomalie)	53,3 min (T > 40 °C)	52,4 min

Rejeu de l'historique réel (2 865 mesures, 11:40 → 14:49, heure de Paris) : 7 incidents ou escalades, dont 6 `gaz_fumee` correspondant aux essais de gaz réalisés, et 10 épisodes de pré-alerte.

Limites. L'historique normal réel couvre quelques heures d'une seule journée et le holdout normal seulement 0,11 h : le taux de faux positifs reste à confirmer sur une durée plus longue. Les incidents `thermique` et `anomalie` du rejeu sont à traiter comme « à vérifier ». Réentraîner après plusieurs heures de fonctionnement normal en étiquetant les essais.

4.2 Vision — YOLOv8n

Le service `sentinel-vision` (`systemd` utilisateur, `vision/vision_service.py`) lit la webcam UGREEN par son chemin stable `/dev/v4l/by-id/...`, capture en 1280×720 MJPEG puis redimensionne en **782×440** avant l'inférence. Ce bridage en amont tient la contrainte du sujet (moins de 100 ms par trame en moyenne). YOLOv8n tourne sur CPU, limité à la classe « personne » avec une confiance minimale de 0,45 ; l'inférence est cadencée à 0,20 s au plus (`SENTINEL_INFER_INTERVAL`, ≈ 5 inférences/s).

Inférence moyenne (30 trames) 41,8 ms	Dernière trame 34,3 ms	Pic depuis démarrage (≈ 16 min) 118 ms	Capacité / flux 23,9 · 10,0 inf/s · FPS	Résolution 782×440
---	----------------------------------	--	--	------------------------------

Relevé sur `/health` du service `vision`, 6 octobre 2026 à 11:43. La capacité est l'inverse du temps moyen d'inférence.

Le temps d'inférence s'affiche en incrustation sur le flux et dans le bloc « Vision IA » du dashboard (`infer_ms`). Sur `person_detected`, le service publie une alerte (`POST /api/v1/alerts`, Bearer). Quand le portillon d'identification

est actif (cas nominal), une simple personne ne déclenche plus la sirène : c'est l'issue du portillon qui décide (§4.4). Sans portillon, l'« Alarme présence » armée déclenche la sirène `intrus` pendant 15 s. Pendant une alerte gaz, la sirène `intrus` est inhibée.

4.3 Reconnaissance faciale et vivacité

La reconnaissance tourne dans le même service, sur CPU, avec les modèles ONNX d'OpenCV (fichiers hors git, téléchargés par `vision/fetch_face_models.py`). C'est une démonstration d'atelier, pas une biométrie de production.

Étape	Modèle	Fonctionnement et paramètres	Code
Détection	YuNet (<code>face_detection_yunet_2023mar</code>)	<code>cv2.FaceDetectorYN</code> , score $\geq 0,6$, NMS 0,3 ; passe visage toutes les 0,30 s	<code>face_engine.py</code>
Empreinte	SFace (<code>face_recognition_sface_2021dec</code>)	visage aligné → vecteur d'empreinte	<code>face_engine.py</code>
Identification	similarité cosinus	comparaison avec chaque photo enrôlée , la meilleure correspondance l'emporte ; « connu » si similarité $\geq 0,363$ (seuil de référence OpenCV SFace, <code>SENTINEL_FACE_THRESHOLD</code>)	<code>face_gallery.py</code>
Vivacité (anti-photo)	MiniFASNetV2 (échelle 2,7) + MiniFASNetV1SE (4,0)	softmax 3 classes [photo, vrai, écran] moyennée ; score vivant = probabilité « vrai visage » ; lissage par piste (moyenne exponentielle $\alpha = 0,4$, suivi IoU) ; vivant si $\geq 0,60$ (2 mesures au moins), leurre si $\leq 0,30$ (4 mesures) ; visages ≥ 48 px ; $\approx 4,3$ ms par visage	<code>face_liveness.py</code>
Verdict	—	connu et vivant → « connu » ; connu mais preuve insuffisante → « vérification » ; leurre → traité comme une intrusion	<code>face_liveness.py</code>

Constats sur site (webcam UGREEN 440p, `docs/preuves-securite.txt`) : vrais visages entre 0,97 et 1,00 en médiane par piste ; visage très proche (> 130 px) : creux ponctuels, moyenne lissée minimale 0,44, d'où le seuil leurre à 0,30 et le seuil vivant à 0,60. Échantillons « faux » du jeu d'origine : 0,03 et 0,09 (rejetés).

Limites. Éclairage, angle et ressemblance peuvent tromper SFace. Visage de moins de 48 px : jamais « connu ». L'anti-spoofing passif peut être trompé par une attaque de haute qualité (masque 3D, vidéo plein écran, grande impression).

4.4 Scénario d'identification (portillon)

Machine d'états pure de `vision/identity.py` (repos → identification → autorisé / intrusion), testée avec une horloge factice ; actions côté API dans `backend/app/face_gate.py` et `extras.py`.

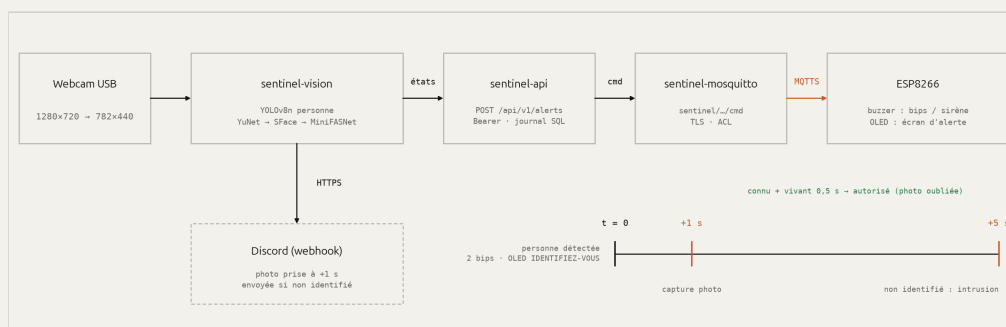


Fig. 4 – Flux du portillon : vision → API → MQTT cmd → ESP ; vision → Discord si la personne n'est pas identifiée.

- Détection.** Une personne (YOLO ou visage) entre dans le champ : la vision publie `identify_start`. L'API envoie sur `sentinel/sentinel-node-01/cmd` deux bips courts « identifiez-vous » et la commande `display identify` : le firmware affiche « ATTENTION / IDENTIFIEZ-VOUS » en plein écran inversé clignotant pendant la fenêtre.
- Fenêtre de 5 s** (`SENTINEL_IDENTIFY_WINDOW_S`). La photo est capturée **1 s après la détection** (`SENTINEL_CAPTURE_DELAY_S`), quand la personne est bien cadrée.
- Identifié.** Un visage connu et vivant, stable 0,5 s, suffit, même avec un inconnu en arrière-plan : bip court, LED verte 2 s, écran « ACCÈS AUTORISÉ » + nom. La photo est **supprimée**, rien n'est envoyé.
- Non identifié.** Fenêtre expirée alors que la personne est toujours là (inconnu, pas de visage, leurre) : état intrusion (ou `intrusion_spoof`). La photo part sur le **webhook Discord** (heure, similarité max, score vivant, caméra ; un envoi toutes les 30 s au plus), l'événement est inscrit au journal (PostgreSQL, dashboard) et l'écran passe en « INTRUS / ALARME ».
- Fin.** Plus personne depuis 5 s : retour au repos et à l'écran normal. Une présence continue ne relance pas les bips.

Interrupteur « Alarme reconnaissance »
(onglets Réglages et Visages)

	Armée	Désarmée
Bips « identifiez-vous », écran OLED	oui	oui
Photo Discord si non identifié	oui	oui
Événement au journal	oui	oui
Sirène intrus (15 s) si non identifié	oui (coupée si la personne s'identifie ensuite)	non

Réglage persistant en base (`face_alarm_enabled`), synchronisé entre les deux onglets. Pendant une alerte gaz, pas de sirène intrus. L'URL du webhook n'est ni versionnée ni journalisée.

4.5 Enrôlement et badges d'accès

- **Onglet Visages** : un nom, puis « **Capturer depuis la caméra** » (trame brute, une photo par clic, en variant les angles) ou des fichiers. JPEG, PNG ou WebP vérifiés par signature, 5 photos au plus par envoi ; une image jugée non vivante (score < 0,60) est refusée.
- **Badge d'accès** : carte par personne avec photo d'identité 3:4 et code ID stable ; « Photo badge » la refait depuis la caméra sans toucher aux empreintes.
- **Stockage** : galerie sur l'hôte (`vision/data/gallery/`, hors git), atteinte via `/api/v1/faces` (session) puis Bearer vers la vision.

05 Rapport d'audit post-pentest (jeudi 08/10)

Modèle à compléter le jeudi 8 octobre 2026. Aucun résultat n'est pré-rempli : les tableaux d'attaques restent vides jusqu'au pentest croisé. Seules les défenses en place (§5.3) décrivent l'état réel au 6 octobre 2026.

Champ	Valeur
Date et créneau	jeudi 8 octobre 2026, après-midi — horaires : _____
Rédacteurs	_____
Groupes adverses	_____
Règles d'engagement (coachs)	à compléter

5.1 Périmètre et cibles

Cible défendue	Adresse	Ports / services	Remarque
PC serveur (Option B)	172.20.10.4	22, 80, 443, 8883	stack Docker + vision ; le reste est filtré
Nœud ESP8266	172.20.10.2	client MQTTS sortant	OTA Wi-Fi protégée par mot de passe si le firmware OTA est flashé (à vérifier le jour J)
Réseau de table	172.20.10.0/28	hotspot « Guiggzz »	accès Wi-Fi des attaquants : à compléter

Cible attaquée (groupe)	Adresse	Ports découverts	Remarque

5.2 Outils

Outil	Usage prévu	Utilisé (cible, heure)
Nmap	découverte d'hôtes et de ports, versions de services (-sT -sV), contrôle de notre propre surface	
Wireshark	capture sur le hotspot : vérifier que MQTT et HTTP sont chiffrés, repérer les flux en clair chez les cibles	
Metasploit	modules auxiliaires de scan et de brute force sur les services exposés	
openssl, curl, client MQTT	versions TLS, en-têtes HTTP, tentatives de connexion MQTT anonymes	

5.3 Défenses en place au début du pentest

- Surface : 22 (SSH par clé, limit), 80 (redirection 308), 443 (HTTPS), 8883 (MQTTs depuis le sous-réseau de table) ; UFW deny par défaut, DOCKER-USER, Samba / Apache / healthai-* arrêtés.
- MQTT : TLS seul, anonyme refusé, ACL par utilisateur, pin du certificat côté ESP, limites de connexions et de messages.
- Web : session HttpOnly/Secure/SameSite=Strict, verrouillage du login (5 échecs / 15 min), contrôle d'Origin (CSRF), CSP stricte, /docs désactivé, entrées validées.

- Caméra : flux uniquement via /cam/* authentifié ; portillon d'identification avec anti-spoofing.
- Conteneurs : cap_drop ALL, no-new-privileges, read_only (api, ml, proxy), limites de ressources.
- Détection : journal sécurité de l'API (LOGIN_FAIL, LOGIN_LOCKOUT, CSRF_BLOCK), connexions refusées de Mosquitto, ss -tnp, journal UFW.

5.4 Attaques subies

Une ligne par tentative observée. Résultat : bloquée / partielle / réussie. Détection : source et heure.

#	Heure	Vecteur	Outil	Résultat	Détection	Correctif
1						
2						
3						
4						
5						
6						

5.5 Attaques menées

#	Heure	Cible	Vecteur	Outil	Résultat	Preuve / recommandation
1						
2						
3						
4						
5						
6						

5.6 Conclusions

BILAN DÉFENSIF

à compléter

BILAN OFFENSIF

à compléter

CORRECTIFS APPLIQUÉS APRÈS LE PENTEST

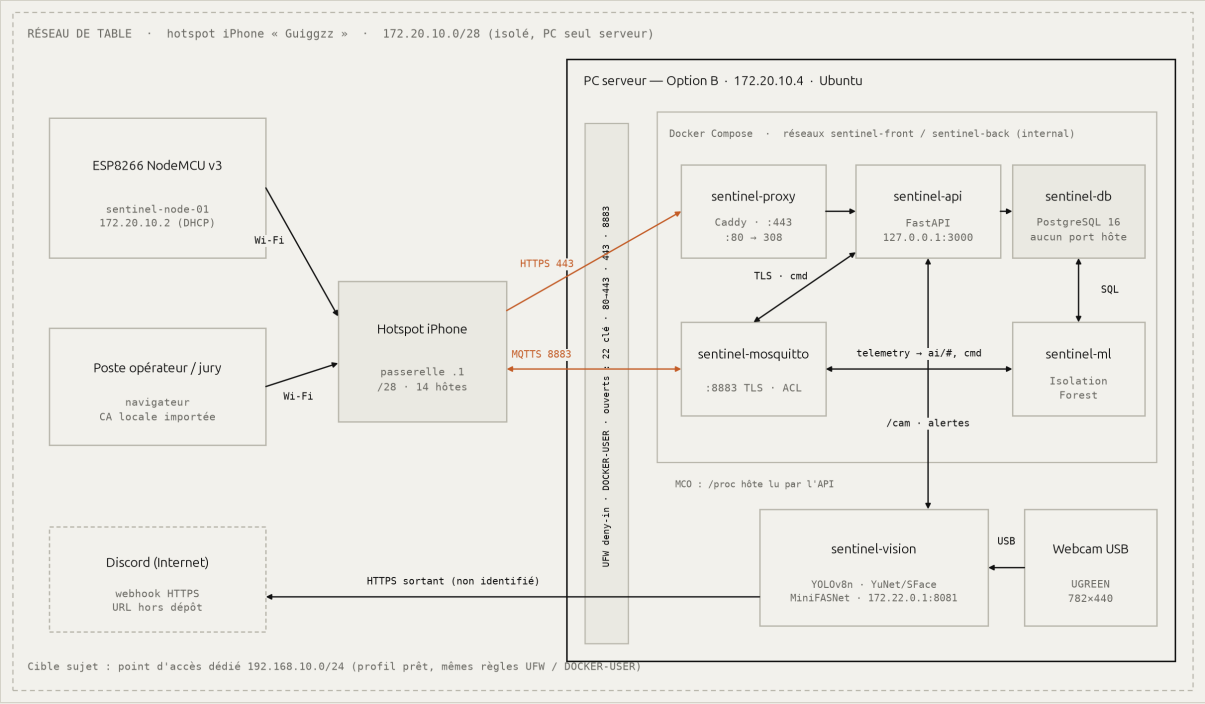
à compléter

ACTIONS RESTANTES

à compléter

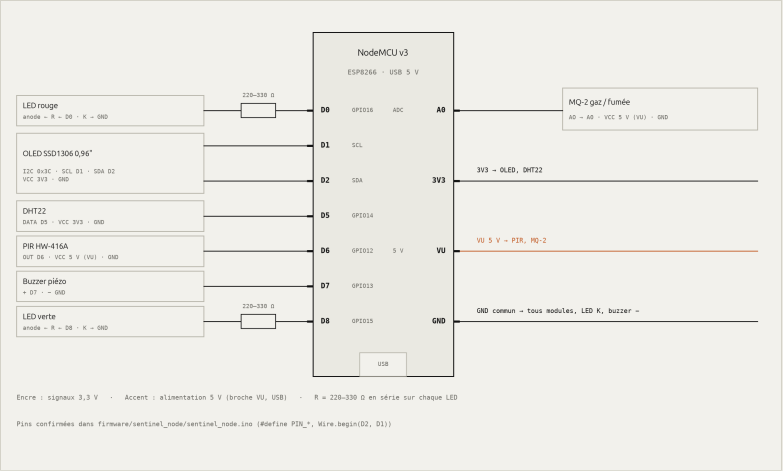
Rappel spray butane 20/20 alarme en 8 s (médiane)	Avance sur règle statique 11–52 min 4/4 dérives lentes	Faux incidents 0 holdout normal 0,11 h	Inférence YOLOv8n 42 ms CPU · 782×440 · moyenne < 100 ms	Identification 5 s SFace 0,363 · vivant ≥ 0,60	Ports exposés 4 22 clé · 80→443 · 443 · 8883
--	---	---	--	---	---

ARCHITECTURE



ESP8266 → MQTTS → Mosquitto → API FastAPI → PostgreSQL / WebSocket → dashboard HTTPS. Le service IA s'abonne à la télémétrie et pilote la sirène. La vision tourne sur le PC, à côté de la webcam : YOLO, portillon d'identification, alerte Discord.

NŒUD DE TABLE



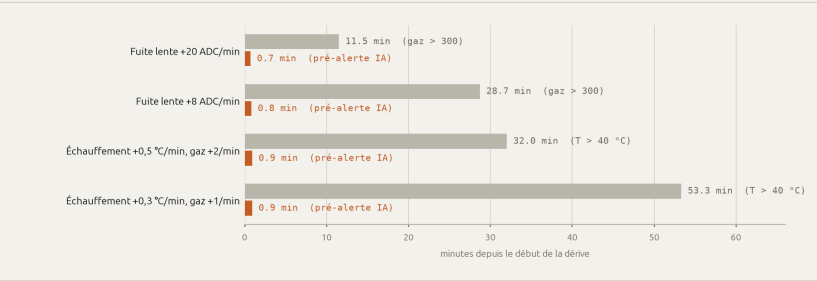
Capteur / actionneur	Broche
DHT22 · OLED I2C	D5 · D1/D2
PIR HW-416A · MQ-2	D6 · A0 (5 V)
Buzzer · LED rouge · LED verte	D7 · D0 · D8

SÉCURITÉ

- MQTTS + pin du certificat, ACL par utilisateur, anonyme refusé
- HTTPS Caddy, session HttpOnly, anti brute-force, CSRF, CSP stricte
- UFW verrouillé + DOCKER-USER, SSH par clé, Samba/Apache arrêtés
- Conteneurs cap_drop ALL, read-only, caméra authentifiée

IA PRÉDICTIVE

Isolation Forest, 300 arbres, 8 features cinétiques directionnelles. Pas de seuil fixe : le risque 0-100 est calibré sur les quantiles ap- pris du score.



RÉPONSE

prediction	pré-alerte silencieuse
gaz_fumee	sirène gaz + LED rouge
thermique	LED rouge + bandeau
identification	bips + OLED, 5 s pour s'identifier
intrusion	Discord + journal ; sirène si armée

01 · CAPTER

DHT22, MQ-2, PIR, OLED d'état ; webcam USB sur le PC serveur

02 · TRANSPORTER

MQTTs 8883, pin du certificat, comptes et ACL par utilisateur

03 · ANALYSER

Isolation Forest ; YOLOv8n ≈ 42 ms ; YuNet + SFace, vivacité MiniFASNet

04 · ALERTER

sirène gaz / intrus, OLED, Discord, dashboard HTTPS temps réel, journal PostgreSQL